SOFTWARE ENGINEERING · CHAPTER 10

Sequence Diagrams

Sequence diagrams describe interactions between actors and objects in time order. They clarify responsibilities and help design service, controller, and component collaborations.

UML 2.5.1 | Mermaid 11 sequenceDiagram | Spring Boot 4.1 · JDK 25 LTS

AGENDA

What we will cover

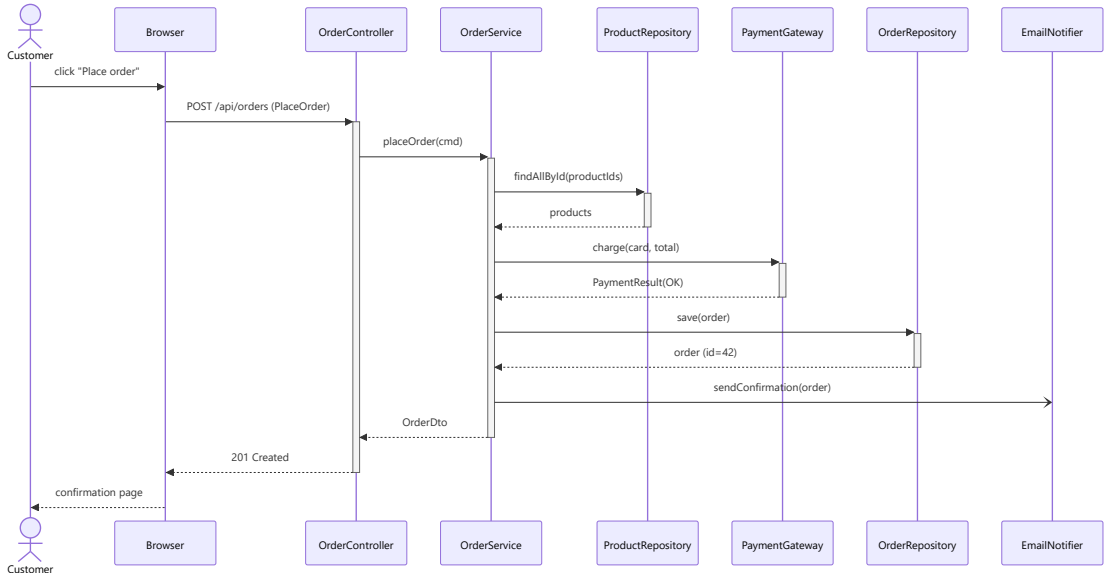
1. Lifelines and activation bars
2. Synchronous and asynchronous messages
3. Calls, returns, and object creation or destruction
4. Actors, boundaries, controls, and entities
5. Conditions, loops, and optional fragments
6. Alternative and parallel interaction fragments
7. Self-calls and callbacks
8. Mapping requirements to interactions
9. Designing APIs and error flows
10. Consistency with class and activity diagrams
11. Chapter Quiz
12. Lab-10

Click any item to jump directly to that topic.

Why sequence diagrams? one scenario, in time order

- **Class diagrams** (Ch. 08) show *who knows whom*; **activity diagrams** (Ch. 09) show *what happens*. A **sequence diagram** shows *who talks to whom, in which order*, for one scenario.
- Time flows **top to bottom**; participants sit left to right in the order they are first used.
- Each message is a method call, a return value, an event or an HTTP request. A good diagram can be typed into a Mockito `InOrder` test.
- Running example: **place order** in the online shop from Chapters 02 to 06: `Browser → OrderController → OrderService → repositories / PaymentGateway → EmailNotifier`.

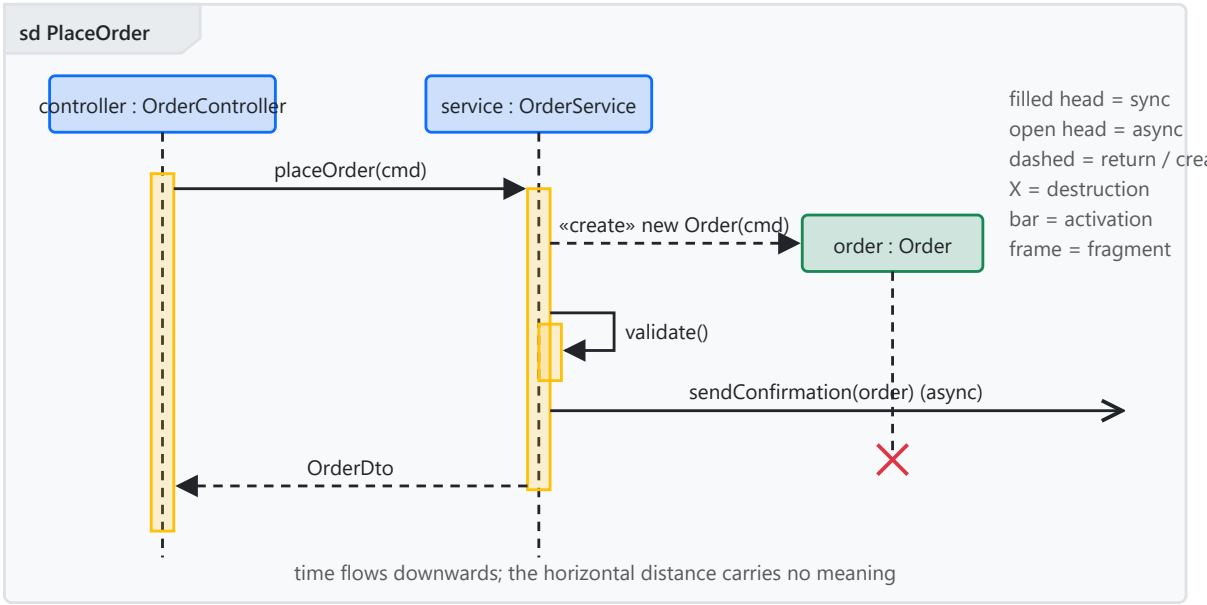
> One diagram = one scenario (happy path, or one failure). Do not try to draw every branch of a use case in a single picture.



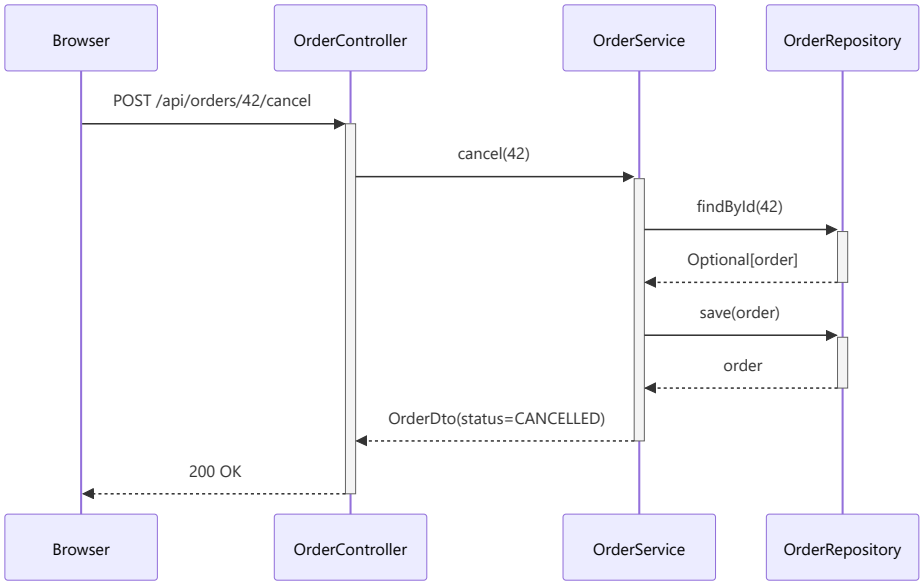
TOPIC 1

The notation in one picture UML 2.5.1

- **Lifeline**: a participant, drawn as a head box and a dashed vertical line. Label `name : Class`, e.g. `orderService : OrderService`; underlining is no longer used in UML 2.
- **Activation bar** (execution specification): the thin rectangle on the lifeline while the object is executing a method. Nested bars = re-entrant or self calls.
- **Message**: horizontal arrow from sender to receiver, labelled with the operation and arguments.
- **Frame**: `sd PlaceOrder` in the top-left corner names the interaction; combined fragments (Topic 5) reuse the frame shape.



A synchronous call: activation on +, return on -



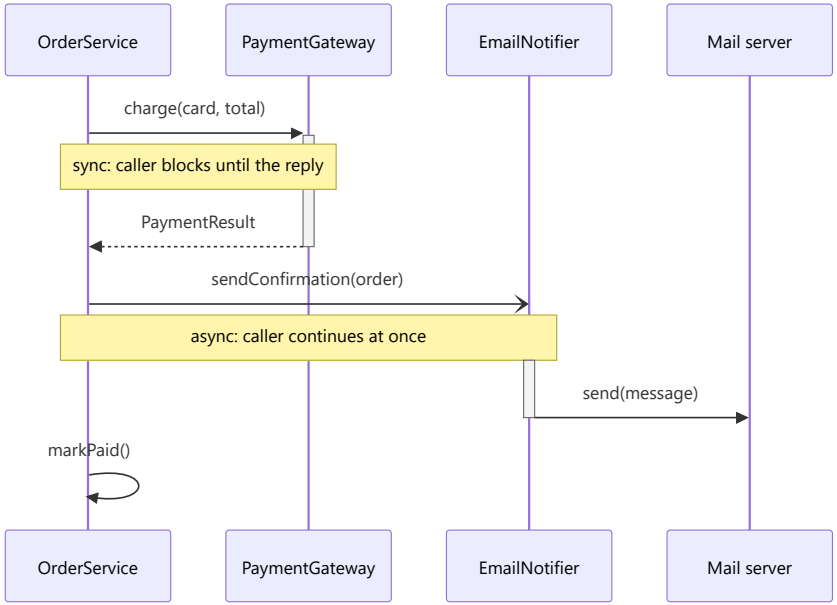

```
sequenceDiagram
  participant B as Browser
  participant OC as OrderController
  participant OS as OrderService
  participant OR as OrderRepository
  B->>+OC: POST /api/orders/42/cancel
  OC->>+OS: cancel(42)
  OS->>+OR: findById(42)
  OR-->>-OS: Optional[order]
  OS->>+OR: save(order)
  OR-->>-OS: order
  OS-->>-OC: OrderDto(status=CANCELLED)
  OC-->>-B: 200 OK
```

```java
@PostMapping("/{id}/cancel")
public OrderDto cancel(@PathVariable long id) {
    // the message label IS the method name
    return orderService.cancel(id);
}

@Transactional
public OrderDto cancel(long id) {
    Order order = orders.findById(id)
        .orElseThrow(() -> new OrderNotFoundException(id));
    order.cancel();
    return OrderDto.from(orders.save(order));
}
```

`+` after the arrow activates the receiver, `-` on the return deactivates it. Every activation must be closed, exactly like every method call must return.

Synchronous vs asynchronous messages

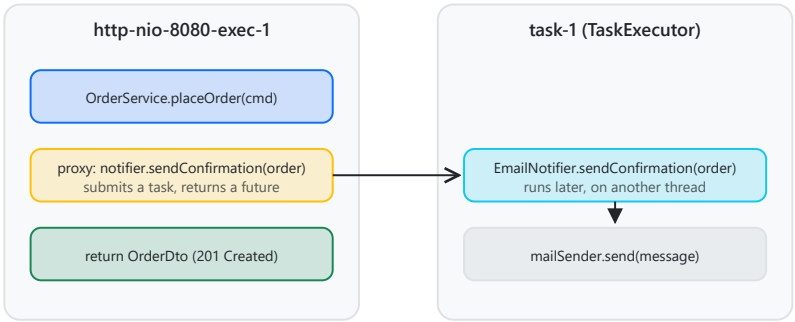


Message	Arrow	Mermaid	Java
Synchronous call	solid, filled head	`A->>B: op()`	method call, caller waits
Asynchronous signal	solid, open head	`A-)B: ev()`	`@Async`, queue, event
Reply (return)	dashed, open head	`B-->>A: value`	return value
Create	dashed to head box	`create participant`	`new X()`
Delete	arrow ending in X	`destroy X`	close, invalidate, GC
Lost / found	to / from a dot	(not supported)	fire-and-forget, timer

- **Synchronous** (filled arrow head): the sender *waits* for the operation to finish. The receiver's activation lasts until the dashed reply. Almost every Java method call is synchronous.
- **Asynchronous** (open arrow head): the sender continues immediately. No reply arrow; if a result comes back later it is a *new message* (callback, Topic 7).
- **Reply**: dashed arrow back to the caller, labelled with the returned value or `result = op()`. Omit trivial `void` replies to reduce clutter, but keep the ones that carry data.
- UML 2.5.1 wording: *synchCall*, *asynchCall*, *asynchSignal*, *reply*, *createMessage*, *deleteMessage*.

```
sequenceDiagram
  participant OS as OrderService
  participant PG as PaymentGateway
  participant EN as EmailNotifier
  participant SMTP as Mail server
  OS->>+PG: charge(card, total)
  Note over OS,PG: sync: caller blocks until the reply
  PG-->>-OS: PaymentResult
  OS-)EN: sendConfirmation(order)
  Note over OS,EN: async: caller continues at once
  activate EN
  EN->>SMTP: send(message)
  deactivate EN
  OS->>OS: markPaid()
```

The open arrow head in Java: `@Async` and `CompletableFuture`



Same proxy mechanism as @Transactional (Ch. 02): self-invocation bypasses @Async.

```java
@Configuration
@EnableAsync
public class AsyncConfig { }

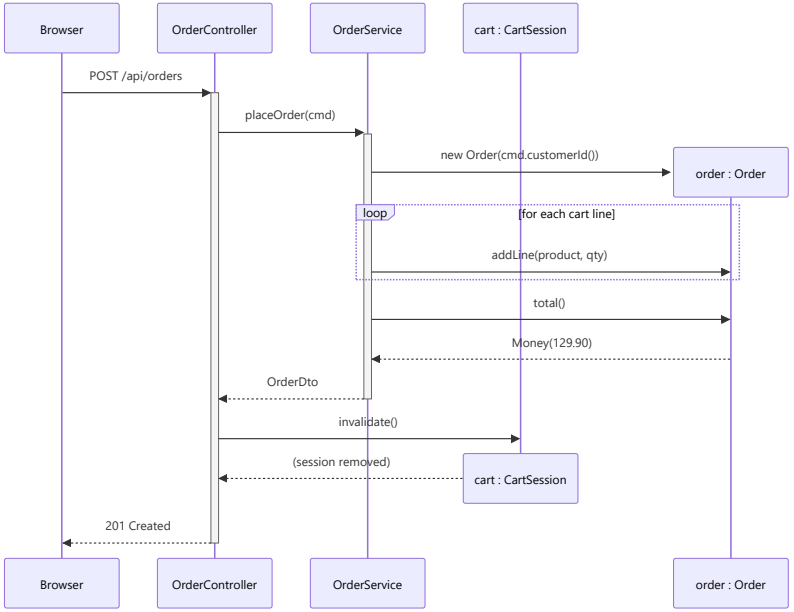
@Component
public class EmailNotifier {
    private final JavaMailSender mailSender;
    public EmailNotifier(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    // asynchronous message: the caller does not wait
    @Async
    public CompletableFuture<Void> sendConfirmation(Order order) {
        var msg = new SimpleMailMessage();
        msg.setTo(order.customerEmail());
        msg.setSubject("Order " + order.id() + " confirmed");
        mailSender.send(msg);
        return CompletableFuture.completedFuture(null);
    }
}

@Service
public class OrderService {
    // ...
    @Transactional
    public OrderDto placeOrder(PlaceOrder cmd) {
        Order order = orders.save(Order.from(cmd));
        // open arrow head: returns immediately with a future
        notifier.sendConfirmation(order);
        return OrderDto.from(order);
    }
}
```

⚠ An `@Async` method that returns `void` swallows exceptions; return a `CompletableFuture` or register an `AsyncUncaughtExceptionHandler`. In the diagram, model the failure as a separate callback message, never as a reply.

Calls, returns, «create» and destruction




```
sequenceDiagram
  participant B as Browser
  participant OC as OrderController
  participant OS as OrderService
  participant Cart as cart : CartSession
  B->>+OC: POST /api/orders
  OC->>+OS: placeOrder(cmd)
  create participant O as order : Order
  OS->>O: new Order(cmd.customerId())
  loop for each cart line
    OS->>O: addLine(product, qty)
  end
  OS->>O: total()
  O-->>OS: Money(129.90)
  OS-->>-OC: OrderDto
  OC->>Cart: invalidate()
  destroy Cart
  Cart-->>OC: (session removed)
  OC-->>-B: 201 Created
```

- **Creation**: the head box appears *at the point in time* where `new` is called; the dashed create message points at the box.
- **Destruction**: a large X ends the lifeline. In Java this is rarely the garbage collector: model `close()`, `invalidate()`, `delete()`.

Every arrow maps to one Java construct

```java
@Entity
public class Order {
    @Id @GeneratedValue private Long id;
    private Long customerId;
    private OrderStatus status = OrderStatus.NEW;
    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    private final List<OrderLine> lines = new ArrayList<>();

    protected Order() { }
    // «create» message
    public Order(Long customerId) { this.customerId = customerId; }

    // synchronous call without an interesting reply (void)
    public void addLine(Product product, int qty) {
        lines.add(new OrderLine(product.id(), product.price(), qty));
    }

    // synchronous call whose reply carries data
    public Money total() {
        return lines.stream().map(OrderLine::subtotal)
                    .reduce(Money.ZERO, Money::plus);
    }
}

@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderService orderService;
    private final CartSession cart;      // @SessionScope bean
    // ...
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public OrderDto placeOrder(@RequestBody @Valid PlaceOrder cmd) {
        OrderDto dto = orderService.placeOrder(cmd);
        // destroy message: the session-scoped cart goes away
        cart.invalidate();
        return dto;
    }
}
```

Diagram element	Java
Lifeline `order : Order`	a variable holding an instance
Sync message `addLine(p, q)`	`order.addLine(p, q);`
Reply `Money(129.90)`	`return` value of `total()`
`«create»` message	constructor call `new Order(id)`
Destruction X	`invalidate()`, `close()`, try-with-resources
Activation bar	stack frame of the method
Nested activation	the method calls another method on the same object

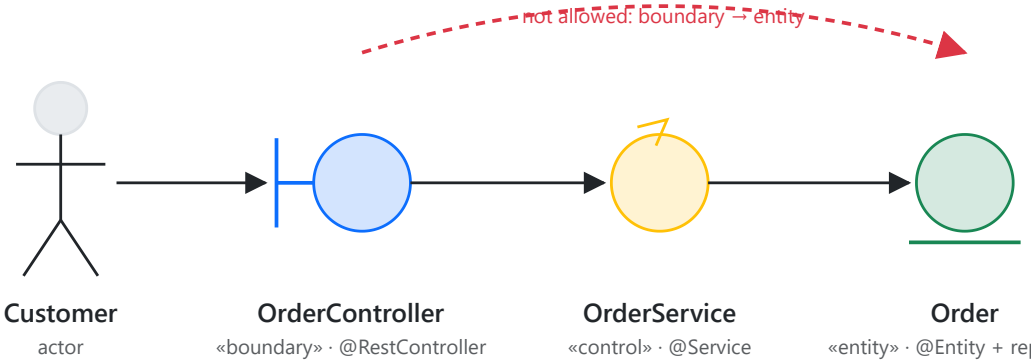
💡 Name messages exactly like the Java operation: `findById(42)`, not "look up the order". Then the diagram doubles as the method list of the receiver's class (Topic 10).

⚠ A reply arrow must return to the *caller*. An arrow from `OrderRepository` straight to the browser is a classic mistake.

TOPIC 4

Actors, boundaries, controls and entities ECB pattern

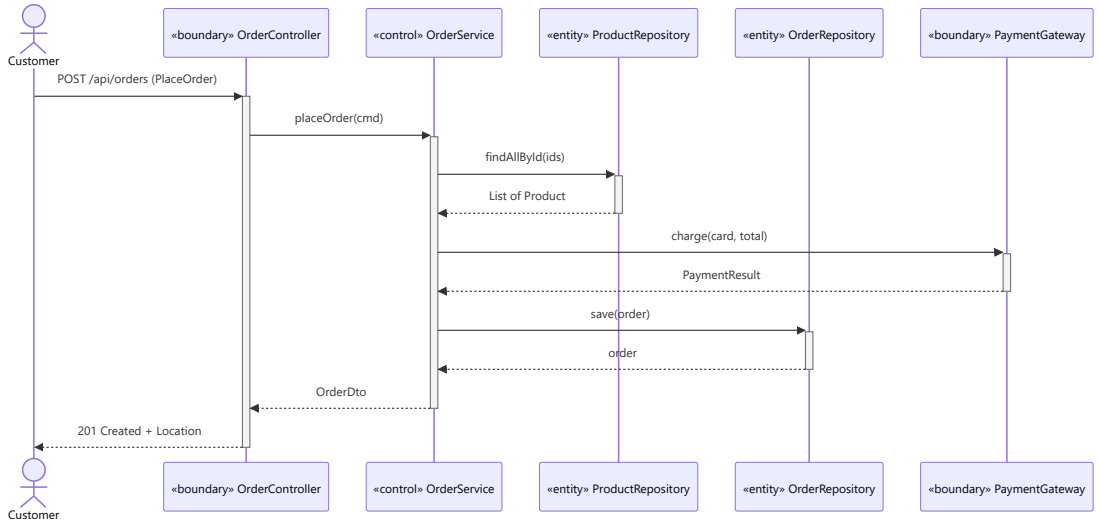
- **Actor**: a person or external system outside our software (Customer, Payment provider). Stick figure, never a Java class.
- **«boundary»**: everything the actor touches: web page, REST controller, adapter to an external API. Spring: `@RestController`, `@Controller`, a `RestClient` wrapper.
- **«control»**: the use-case logic that coordinates. Spring: `@Service`.
- **«entity»**: long-lived domain data. Spring: `@Entity` classes plus their `@Repository` gateways.
- **Rules**: actors talk only to boundaries; boundaries never talk to entities directly; entities never call boundaries or actors.



Robustness analysis (Jacobson): one boundary per screen or endpoint, one control per use case, entities from the domain model.

A control may call another control (OrderService → PricingService) and several entities.

Place order, ECB version: one boundary, one control, entities behind gateways




```
sequenceDiagram
  actor C as Customer
  participant OC as «boundary» OrderController
  participant OS as «control» OrderService
  participant PR as «entity» ProductRepository
  participant OR as «entity» OrderRepository
  participant PG as «boundary» PaymentGateway
  C->>+OC: POST /api/orders (PlaceOrder)
  OC->>+OS: placeOrder(cmd)
  OS->>+PR: findAllById(ids)
  ...
```

```java
@RestController                     // «boundary»
public class OrderController { ... }

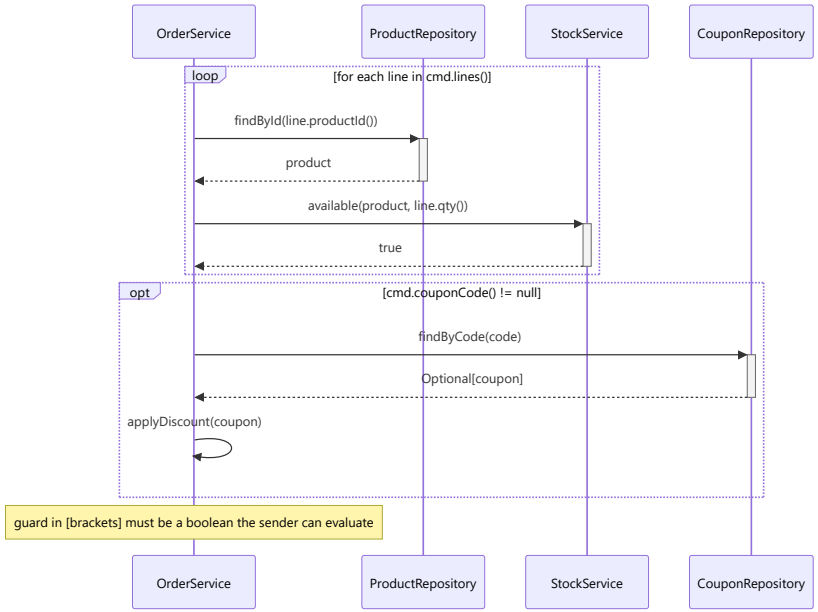
@Service                            // «control»
public class OrderService { ... }

@Entity                             // «entity»
public class Order { ... }

public interface OrderRepository    // «entity» gateway
        extends JpaRepository<Order, Long> { }

@Component                          // «boundary» to Stripe
public class PaymentGateway {
    private final RestClient client;
    public PaymentResult charge(Card card, Money total) {
        return client.post().uri("/v1/charges")
            .body(new ChargeRequest(card, total))
            .retrieve().body(PaymentResult.class);
    }
}
```

Combined fragments: `loop` and `opt` with guards




```
sequenceDiagram
  participant OS as OrderService
  participant PR as ProductRepository
  participant SS as StockService
  participant CR as CouponRepository
  loop for each line in cmd.lines()
    OS->>+PR: findById(line.productId())
    PR-->>-OS: product
    OS->>+SS: available(product, line.qty())
    SS-->>-OS: true
  end
  opt cmd.couponCode() != null
    OS->>+CR: findByCode(code)
    CR-->>-OS: Optional[coupon]
    OS->>OS: applyDiscount(coupon)
  end
```

- **Fragment** = a frame with an *operator* in the pentagon and one or more *operands* separated by dashed lines.
- **Guard** `[condition]` sits at the top-left of the operand. `loop [min, max]` or `loop [while stock > 0]`.
- `opt` is an `alt` with one operand and no `else`: "do this or nothing".

Fragment operators UML 2.5.1 §17.6 and the Java they stand for

```java
@Transactional
public OrderDto placeOrder(PlaceOrder cmd) {
    Order order = new Order(cmd.customerId());

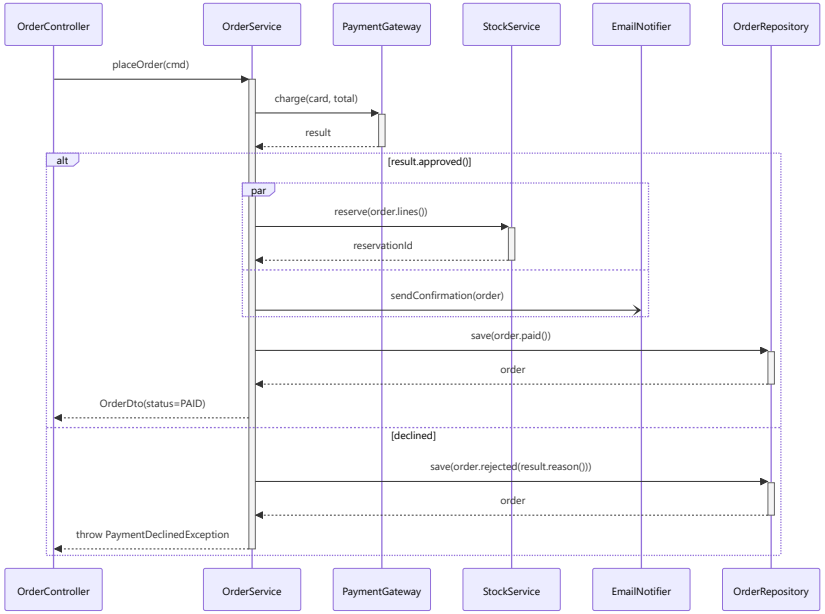
    // loop [for each line in cmd.lines()]
    for (OrderLine line : cmd.lines()) {
        Product product = products.findById(line.productId())
            .orElseThrow(() -> new ProductNotFound(line.productId()));
        if (!stockService.available(product, line.qty())) {
            // break fragment (Topic 9)
            throw new OutOfStockException(product.id());
        }
        order.addLine(product, line.qty());
    }

    // opt [cmd.couponCode() != null]
    if (cmd.couponCode() != null) {
        coupons.findByCode(cmd.couponCode())
               .ifPresent(order::applyDiscount);
    }
    return OrderDto.from(orders.save(order));
}
```

Operator	Meaning	Java	Mermaid
`alt`	mutually exclusive operands, each with a guard; `[else]` allowed	`if / else if / else`, `switch`	`alt … else … end`
`opt`	zero or one execution	`if` without `else`	`opt … end`
`loop`	repeat while guard holds, `[min, max]` bounds	`for`, `while`, stream	`loop … end`
`par`	operands run concurrently, any interleaving	`@Async`, `CompletableFuture.allOf`	`par … and … end`
`break`	if guard holds: run operand, leave enclosing fragment	`throw`, early `return`	`break … end`
`critical`	atomic region, no interleaving from `par`	`synchronized`, `@Transactional`, lock	`critical … end`
`ref`	interaction use: include another diagram	a method call to a documented method	(draw as a Note)
`neg`	invalid trace: what must *not* happen	a test that asserts it never happens	(draw as a Note)

Also in the standard but rarely useful in design work: `seq`, `strict`, `ignore`, `consider`, `assert`.

`alt`: payment OK or declined · `par`: reserve stock while notifying




```
OS->>+PG: charge(card, total)
PG-->>-OS: result
alt result.approved()
  par
    OS->>+SS: reserve(order.lines())
    SS-->>-OS: reservationId
  and
    OS-)EN: sendConfirmation(order)
  end
  OS->>+OR: save(order.paid())
  OR-->>-OS: order
  OS-->>OC: OrderDto(status=PAID)
else declined
  OS->>+OR: save(order.rejected(reason))
  OR-->>-OS: order
  OS-->>-OC: throw PaymentDeclinedException
end
```

- `alt` guards must be **exhaustive and exclusive**; use `[else]` for the remaining case.
- `par` says the operands *may* interleave. It does not promise threads; the implementation decides (Topic 6, next slide).
- Nesting is allowed and common: `par` inside `alt` inside `loop`. Keep it to two levels.

`if/else` mirrors `alt`; `CompletableFuture` mirrors `par`

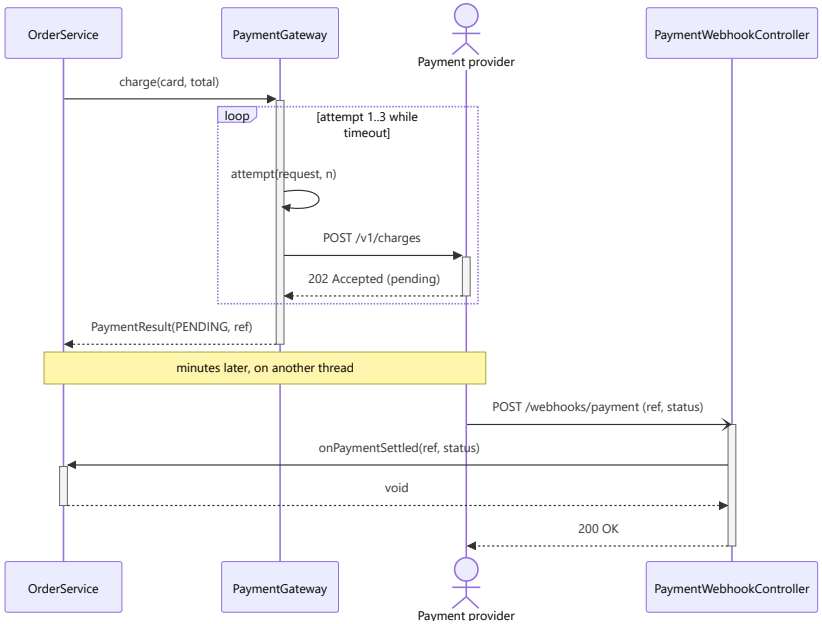
```
@Transactional
public OrderDto placeOrder(PlaceOrder cmd) {
    Order order = buildOrder(cmd);
    PaymentResult result = paymentGateway.charge(cmd.card(), order.total());

    // alt [result.approved()]
    if (result.approved()) {
        // par: reserve stock and notify, in any order
        CompletableFuture<String> reservation =
            CompletableFuture.supplyAsync(
                () -> stockService.reserve(order.lines()), executor);
        CompletableFuture<Void> mail = notifier.sendConfirmation(order);
        CompletableFuture.allOf(reservation, mail).join();
        order.paid(reservation.join());
        return OrderDto.from(orders.save(order));
    }
    // else [declined]
    orders.save(order.rejected(result.reason()));
    throw new PaymentDeclinedException(result.reason());
}
```

Diagram	Implementation choice
`par` with two sync operands	Two futures joined with `allOf` (shown), or a plain sequential call if concurrency is not worth it
`par` with an async operand	Just fire the `@Async` method; nothing to join
`alt` with a `throw` in one operand	Guard clause + exception; the controller advice maps it to a status (Topic 9)
`critical` around `save`	The `@Transactional` boundary or a pessimistic lock

⚠ A `par` whose operands both write the same entity needs a `critical` region or a version column (`@Version`). The diagram makes the race visible; the code must resolve it.

Self-call: retry inside `PaymentGateway` · callback: the provider's webhook




```
OS->>+PG: charge(card, total)
loop attempt 1..3 while timeout
  PG->>PG: attempt(request, n)
  PG->>+Stripe: POST /v1/charges
  Stripe-->>-PG: 202 Accepted (pending)
end
PG-->>-OS: PaymentResult(PENDING, ref)
Note over OS,Stripe: minutes later
Stripe-)WH: POST /webhooks/payment
activate WH
WH->>+OS: onPaymentSettled(ref, status)
OS-->>-WH: void
WH-->>Stripe: 200 OK
deactivate WH
```

- **Self-call** `A->>A`: the object invokes one of its own methods; UML draws a nested activation bar.
- **Callback**: the earlier receiver later sends a *new* message back. It starts its own activation and is never drawn as a reply.
- The `ref` value correlates the two halves; note that the webhook arrives on a different request thread.

Self-call in a retry loop, callback as a webhook endpoint

```java
@Component
public class PaymentGateway {
    private final RestClient client;
    private static final int MAX_ATTEMPTS = 3;

    public PaymentResult charge(Card card, Money total) {
        var request = new ChargeRequest(card, total);
        for (int n = 1; n <= MAX_ATTEMPTS; n++) {
            try {
                // self-call: PG->>PG: attempt(request, n)
                return attempt(request, n);
            } catch (ResourceAccessException timeout) {
                if (n == MAX_ATTEMPTS) throw timeout;
            }
        }
        throw new IllegalStateException("unreachable");
    }

    private PaymentResult attempt(ChargeRequest request, int n) {
        return client.post().uri("/v1/charges")
            .header("Idempotency-Key", request.key())
            .body(request)
            .retrieve()
            .body(PaymentResult.class);
    }
}
```

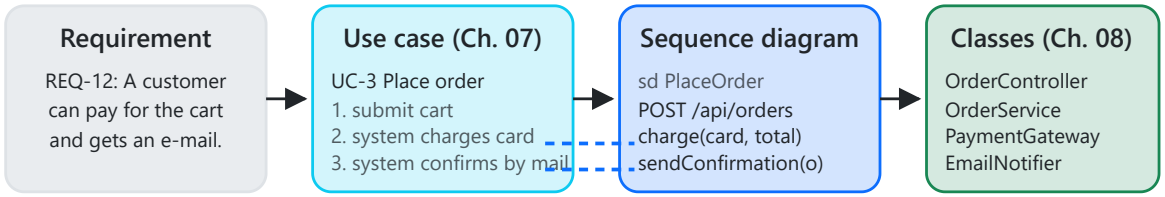
```java
@RestController
@RequestMapping("/webhooks/payment")
public class PaymentWebhookController {
    private final OrderService orderService;
    public PaymentWebhookController(OrderService s) {
        this.orderService = s;
    }

    // callback: a new message from the provider, on a new thread
    @PostMapping
    public ResponseEntity<Void> onEvent(
            @RequestBody PaymentEvent event,
            @RequestHeader("Stripe-Signature") String sig) {
        signatureVerifier.verify(event, sig);
        orderService.onPaymentSettled(event.ref(), event.status());
        return ResponseEntity.ok().build();
    }
}

@Service
public class OrderService {
    @Transactional
    public void onPaymentSettled(String ref, PaymentStatus status) {
        Order order = orders.findByPaymentRef(ref)
            .orElseThrow(() -> new OrderNotFoundException(ref));
        if (status == PaymentStatus.SUCCEEDED) order.paid(ref);
        else order.rejected("declined by provider");
    }
}
```

💡 Webhooks are retried by the provider: make `onPaymentSettled` idempotent (ignore an already-paid order). The `loop` in the diagram should say so in a Note.

Traceability: requirement → use case → sequence → class



Mapping rules

- one sequence diagram per use-case scenario: main success scenario + one per important extension
- "actor does X" → message from the actor to a boundary • "system does Y" → messages control → entity
- every noun in the step that must survive → an entity lifeline • every verb → a message (operation name)
- every extension (3a: card declined) → alt / break operand with the extension text as the guard

Trace links: REQ-12 → UC-3 → sd PlaceOrder → OrderService.placeOrder → OrderServiceTest.placesOrderInDiagramOrder

UC-3 step	Messages in `sd PlaceOrder`
1. Customer submits the cart	`POST /api/orders` → `placeOrder(cmd)`
2. System validates the lines	`loop`: `findById`, `available`
3. System charges the card	`charge(card, total)` → `PaymentResult`
3a. Card declined	`alt [declined]`: `save(rejected)`, exception
4. System stores the order	`save(order)`
5. System confirms by e-mail	`sendConfirmation(order)`, async

If a use-case step has no message, it is either UI detail (drop it) or a missing responsibility (add a lifeline). If a message has no step, ask why it exists.

The scenario is the test: `InOrder` mirrors the arrows top to bottom

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock ProductRepository products;
    @Mock PaymentGateway paymentGateway;
    @Mock OrderRepository orders;
    @Mock EmailNotifier notifier;
    @InjectMocks OrderService service;

    @Test
    void placesOrderInDiagramOrder() {
        var cmd = new PlaceOrder(7L, List.of(new Line(1L, 2)), card());
        when(products.findById(1L)).thenReturn(Optional.of(product(1L)));
        when(paymentGateway.charge(any(), any()))
            .thenReturn(PaymentResult.approved("ref-1"));
        when(orders.save(any())).thenAnswer(inv -> inv.getArgument(0));
        when(notifier.sendConfirmation(any()))
            .thenReturn(CompletableFuture.completedFuture(null));

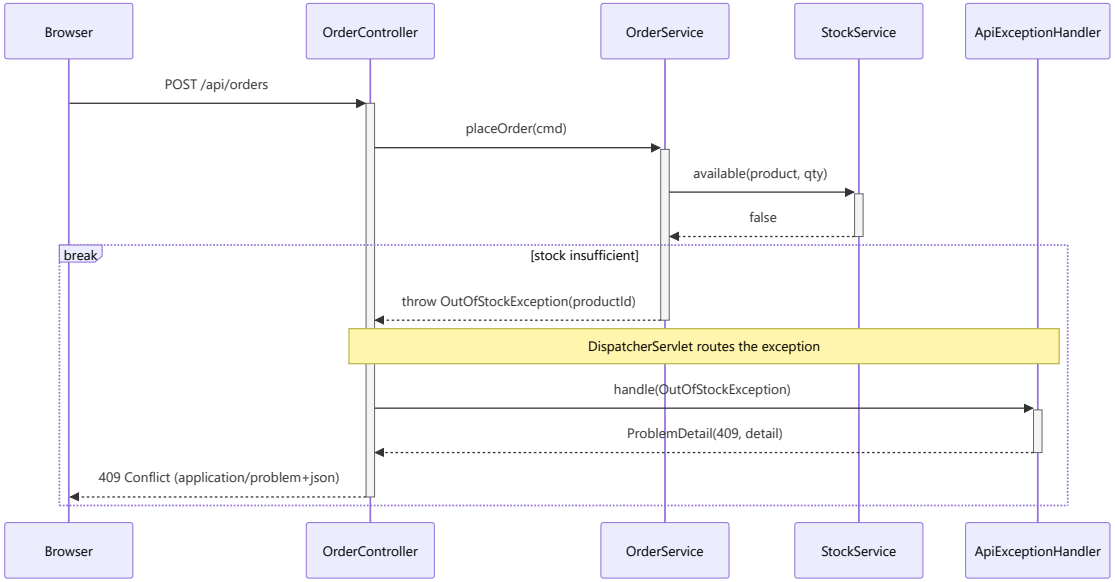
        OrderDto dto = service.placeOrder(cmd);

        // one verify per message, in the order of the diagram
        InOrder seq = inOrder(products, paymentGateway, orders, notifier);
        seq.verify(products).findById(1L);
        seq.verify(paymentGateway).charge(eq(cmd.card()), any(Money.class));
        seq.verify(orders).save(any(Order.class));
        seq.verify(notifier).sendConfirmation(any(Order.class));
        seq.verifyNoMoreInteractions();
        assertThat(dto.status()).isEqualTo(OrderStatus.PAID);
    }
}
```

- **Lifelines → mocks**: every participant except the one under test becomes a `@Mock`.
- **Reply arrows → stubs**: each dashed return becomes a `when(...).thenReturn(...)`.
- **Messages → verify**: `inOrder(...)` checks the vertical order; `verifyNoMoreInteractions()` catches messages that are in the code but not in the diagram.
- **alt operands → test methods**: `placesOrderInDiagramOrder()`, `rejectsDeclinedPayment()`, one test per scenario.

Chapter 06 recap: JUnit 6.1 + Mockito 5.23 via `spring-boot-starter-test`. No Spring context is needed for this test; it runs in milliseconds.

Error flow: exception → `@ExceptionHandler` → `ProblemDetail`




```
sequenceDiagram
  participant B as Browser
  participant OC as OrderController
  participant OS as OrderService
  participant SS as StockService
  participant EH as ApiExceptionHandler
  B->>+OC: POST /api/orders
  OC->>+OS: placeOrder(cmd)
  OS->>+SS: available(product, qty)
  SS-->>-OS: false
  break stock insufficient
    OS-->>-OC: throw OutOfStockException(id)
    OC->>+EH: handle(OutOfStockException)
    EH-->>-OC: ProblemDetail(409, detail)
    OC-->>-B: 409 Conflict
  end
```

- An exception is drawn as a **reply arrow labelled** `throw X` (or a stereotyped «exception» reply); the activation ends early.
- `break` leaves the enclosing interaction, exactly like `throw` leaves the method.
- The handler is a real lifeline: it is where the HTTP status is decided.

Deriving the API: the first message into a boundary is the endpoint

```java
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(OutOfStockException.class)
    ProblemDetail outOfStock(OutOfStockException ex) {
        var pd = ProblemDetail.forStatusAndDetail(
            HttpStatus.CONFLICT, ex.getMessage());
        pd.setTitle("Out of stock");
        pd.setProperty("productId", ex.productId());
        return pd;
    }

    @ExceptionHandler(PaymentDeclinedException.class)
    ProblemDetail declined(PaymentDeclinedException ex) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.UNPROCESSABLE_CONTENT, ex.getMessage());
    }

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        return ProblemDetail.forStatusAndDetail(
            HttpStatus.NOT_FOUND, ex.getMessage());
    }
}

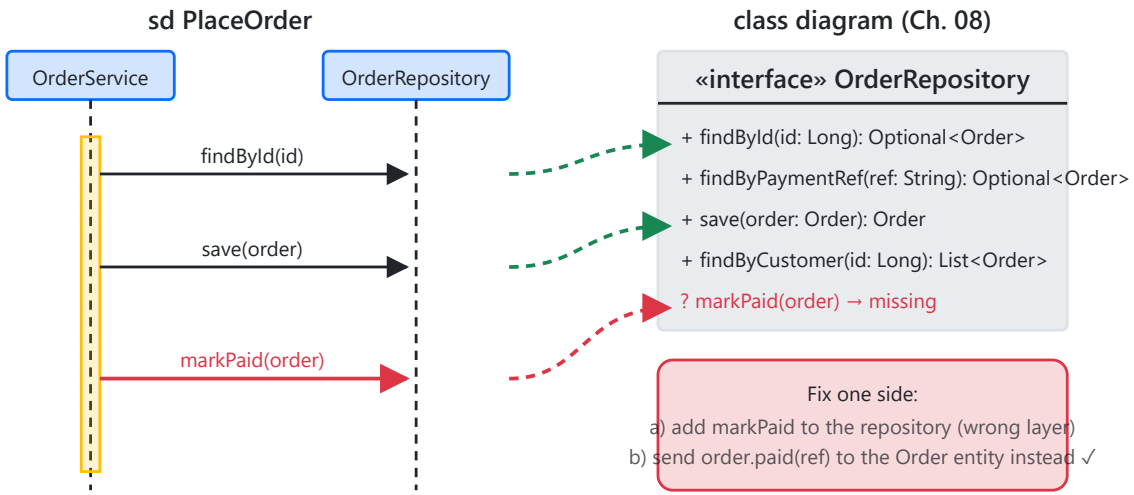
// the try/catch lives in ONE place, the boundary adapter,
// never in the controller
public PaymentResult charge(Card card, Money total) {
    try {
        return attempt(new ChargeRequest(card, total), 1);
    } catch (HttpClientErrorException e) {
        throw new PaymentDeclinedException(e.getResponseBodyAsString());
    }
}
```

Message into the boundary	Endpoint	Failure reply	Status
`placeOrder(cmd)`	`POST /api/orders` → 201	`throw OutOfStock`	409
		`throw PaymentDeclined`	422
		invalid `cmd`	400
`cancel(id)`	`POST /api/orders/{id}/cancel` → 200	`throw OrderNotFound`	404
`onPaymentSettled(ref)`	`POST /webhooks/payment` → 200	bad signature	401
`findById(id)` (query)	`GET /api/orders/{id}` → 200	missing	404

- **Method signature** = message label + reply type: `placeOrder(PlaceOrder): OrderDto`.
- **Request body** = the arguments of the first message; **response body** = its reply.
- Each `alt`/`break` operand that ends at the boundary needs a status code and a `ProblemDetail` shape.

⚠ If the diagram shows the controller catching the exception itself, the design is wrong (Lab-02 rule: no `try/catch` in controllers).

Consistency check: every message is an operation of the receiver's class

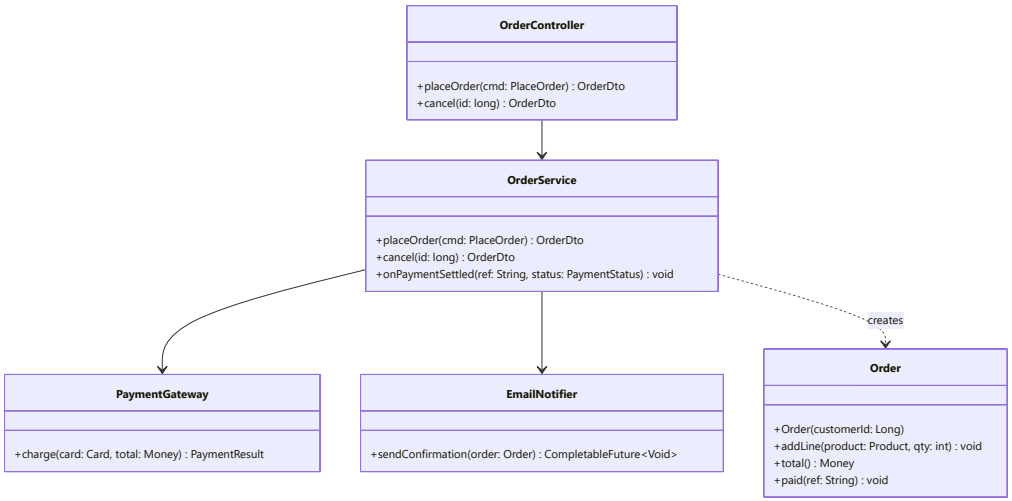


Also check: every lifeline is a class (or actor); replies match return types; guards match activity decision nodes.

Check	Sequence ↔ class diagram
Lifeline	an instance of a class in the class diagram (or an actor)
Message	a public operation of the receiver's class, same name and arity
Reply	the operation's return type (`Optional<Order>`, not `Order`)
Sender → receiver	an association or dependency from the sender's class to the receiver's class must exist
«create»	a constructor with those parameters

Check	Sequence ↔ activity diagram (Ch. 09)
Action node	one or more messages, in the same order
Decision node + guards	`alt` / `opt` operands with the same guards
Fork / join	`par` fragment
Swimlane	usually one lifeline (or one ECB group)

Three artefacts, one vocabulary: sequence, class diagram, Java




```java
// Every message in sd PlaceOrder resolves to one of these.
public interface OrderRepository extends JpaRepository<Order, Long> {
    Optional<Order> findByPaymentRef(String ref);
}

@Service
public class OrderService {
    private final ProductRepository products;
    private final OrderRepository orders;
    private final PaymentGateway paymentGateway;
    private final EmailNotifier notifier;
    private final StockService stockService;

    public OrderService(ProductRepository products,
                        OrderRepository orders,
                        PaymentGateway paymentGateway,
                        EmailNotifier notifier,
                        StockService stockService) {
        // one constructor parameter per outgoing association
        this.products = products; this.orders = orders;
        this.paymentGateway = paymentGateway;
        this.notifier = notifier; this.stockService = stockService;
    }

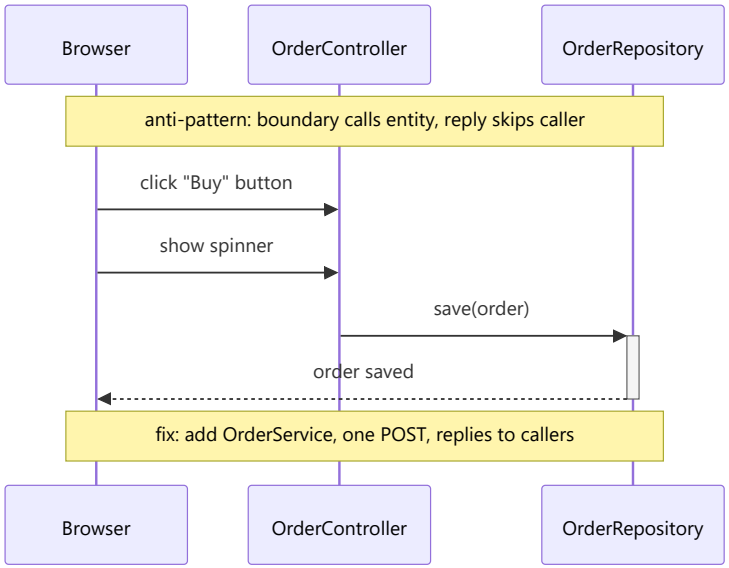
    public OrderDto placeOrder(PlaceOrder cmd) { ... }
    public OrderDto cancel(long id) { ... }
    public void onPaymentSettled(String ref, PaymentStatus s) { ... }
}
```

Review checklist: (1) receiver has the operation, (2) sender holds a reference to the receiver, (3) reply type matches, (4) the same guard text appears in the activity diagram, (5) the test verifies the same order.

REVIEW

Common mistakes and how to fix them

Mistake	Why it hurts	Fix
Arrow from a lifeline to nowhere (or to the frame edge)	No receiver means no class owns the operation; it cannot be implemented or tested.	Add the missing lifeline, or use a *found/lost* message only for real external events (timer, network).
Missing returns where data flows back, or a return for every `void`	Readers cannot tell where the price came from; or the diagram doubles in height with empty dashed arrows.	Draw a reply when it carries a value or ends a long activation; omit `void` replies inside tight loops.
Too many objects (every DTO, every helper, the logger)	Fifteen lifelines cannot be read on one screen; the essential collaboration disappears.	Seven lifelines or fewer; group repositories behind one entity gateway; hide utilities; split into two diagrams with a `ref`.
UI details in a design diagram ("click OK", "show spinner")	The interaction between classes is polluted with presentation events that change every sprint.	One message from the actor to the boundary (`POST /api/orders`); UI behaviour belongs in a separate UI sequence or a state diagram.
Activation bars that never end, or that start without a message	The reader cannot tell which method is running; implementation of the call stack becomes ambiguous.	Every `+` has a `-`; every bar begins with an incoming arrow.
Guards that are not booleans ("payment", "maybe")	The `alt` cannot be turned into an `if`; test cases cannot be derived.	Write the guard as an expression the sender can evaluate: `[result.approved()]`.
Callback drawn as a reply arrow	Suggests the caller was blocked for minutes; hides the second thread and the idempotency problem.	Async message out, new message (with its own activation) back in.
Message names that differ from the class diagram	Two truths: developers implement one, reviewers verify the other.	Copy operation names verbatim; run the consistency table (Topic 10) before the review.



⊗ Three violations in one picture: no control lifeline, a reply that skips the caller, and two UI events instead of one request.

REVIEW

Best practices for design-level sequence diagrams

1 One scenario per diagram

Happy path first, then one diagram per important extension. Name the frame after the use case: `sd PlaceOrder`, `sd PlaceOrder-Declined`.

2 Message label = method name

`findAllById(ids)`, not "get the products". The receiver's class must have that operation, and the test verifies exactly that call.

3 ECB layering

Actor → boundary → control → entity. A boundary that talks to a repository or an entity that sends e-mail is a design smell, not a drawing style.

4 Show the activation, close it

Activation bars make the call stack visible. Every bar starts with a message and ends with a reply or a `throw`.

5 Fragments with boolean guards

`alt` operands exhaustive and exclusive, `loop` with a bound, `break` for errors, `par` only when the code really runs concurrently.

6 Async is a different arrow

Open arrow head, no reply, callback as a new message. In code: `@Async` + `CompletableFuture`, a queue, or a webhook endpoint.

7 Errors end at the boundary

Every failure operand must reach the boundary with a status code and a `ProblemDetail`. The handler is a lifeline, the controller has no `try/catch`.

8 Cross-check the models

Messages vs class operations, guards vs activity decisions, lifelines vs classes. Keep a consistency table in the review, fix mismatches on the correct side.

9 Keep the source, generate the picture

Mermaid or PlantUML text lives in the repository next to the code and is reviewed in the merge request; PNGs are exports, never the master.

Chapter quiz 10 questions

1. In a sequence diagram, what does the thin rectangle drawn on a lifeline represent?

- The class of the object
- An activation: the period during which the object executes a method
- A database transaction
- A combined fragment

2. Which arrow head marks an asynchronous message in UML 2.5?

- Filled (solid) head
- Open (stick) head
- Hollow triangle
- Diamond

3. How is object creation shown?

- A dashed message ending at the head box of a lifeline that starts at that point in time
- A large X on the lifeline
- A bold lifeline
- A note saying "new"

4. In the ECB pattern, which communication is NOT allowed?

- Actor to boundary
- Boundary to control
- Control to entity
- Boundary directly to entity

5. Which fragment operator models "do these messages zero or one time"?

- loop
- alt
- opt
- par

6. What must be true of the guards of an alt fragment?

- They must all be the same expression
- They must be exhaustive and mutually exclusive
- They must reference the actor
- They must be numbers

7. A payment provider calls our webhook minutes after we sent it an async request. How should the webhook call be drawn?

- As the dashed reply of the original async message
- As a new message from the provider to a boundary, with its own activation
- As a self-call on PaymentGateway
- It should not appear

8. Which Mockito feature lets a test verify that mocked collaborators were called in the order shown in the diagram?

- @Spy
- ArgumentCaptor
- InOrder
- @MockitoBean

9. An OrderService message ends in throw OutOfStockException. Where is the HTTP status decided?

- In the OrderService
- In the repository
- In the @ExceptionHandler of a @RestControllerAdvice, returning a ProblemDetail
- In the browser

10. Which check ensures a sequence diagram is consistent with the class diagram?

- Every lifeline has a different colour
- Every message is an operation of the receiver's class and the sender holds a reference to the receiver
- Every message has a reply
- There are at most three lifelines

WRAP-UP

Summary

A sequence diagram shows one scenario: lifelines left to right, time top to bottom, activation bars for running methods.

Filled arrow head = synchronous call with a dashed reply; open arrow head = asynchronous message with no reply (`@Async`, `CompletableFuture`).

`«create»` points at a new head box (constructor); a large X ends a lifeline (`close`, `invalidate`).

ECB: actors talk to boundaries (`@RestController`), boundaries to controls (`@Service`), controls to entities (`@Entity` + repositories).

Fragments: `loop`, `opt`, `alt`/`else`, `par`/`and`, `break`, `critical`, `ref`, `neg`; guards are booleans in `[brackets]`.

Self-calls nest an activation on the same lifeline; callbacks (webhooks) are new messages, never replies.

Use-case steps become messages; extensions become `alt`/`break` operands; the diagram becomes a Mockito `InOrder` test.

The first message into a boundary defines the endpoint; every failure operand needs a status code and a `ProblemDetail` from `@ExceptionHandler`.

Consistency: every message is an operation on the receiver's class, every guard matches the activity diagram, every lifeline is a class or an actor.

Open Lab-10: 5 tasks + 1 challenge

Next chapter: 11 · Component Diagrams